



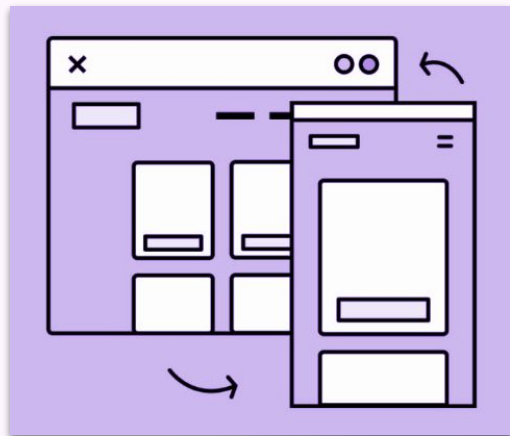
04

Design Responsive

Adaptation des interfaces

Utile ?

Primordiale.



Le **design responsive** est une approche de conception qui vise à rendre une interface **adaptable et cohérente** sur tous les types d'écrans.

Elle repose sur la **restructuration dynamique** de la mise en page et des éléments du contenu afin de préserver l'**ergonomie et la lisibilité** du projet, quel que soit le support utilisé.



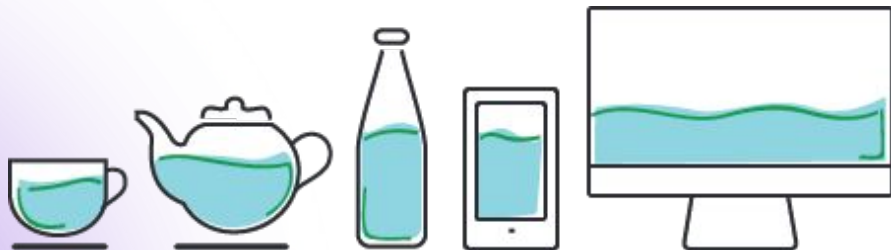
Mobile First

Le verre à moitié rempli

En juillet 2025, les smartphones représentaient 64,35 % du trafic web mondial, selon [SOAX](#).

Il est donc primordial de mettre la priorité sur l'adaptation de votre site web à ce type d'écran.

Pour ce faire, nous avons à notre disposition plusieurs outils et bonnes pratiques pour arriver à nos fins.



Le pourquoi du comment

Principe :

Concevoir d'abord pour le mobile, puis enrichir pour les écrans plus grands.

Avantages de conception :

Performance, accessibilité, hiérarchie visuelle optimisée.

Cela permet d'éviter la surcharge visuelle et d'optimiser la vitesse (facteur clé pour le SEO et la rétention des visiteurs).

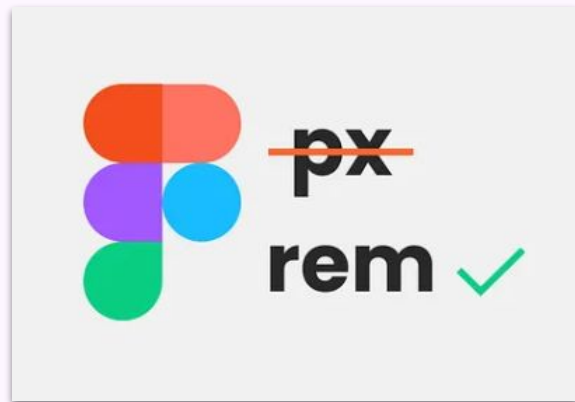
C'est également une manière d'aller à l'essentiel que ce soit dans son contenu graphique ou textuel.

1. Les unités de mesure dynamiques

L'utilisation d'unités relatives telle que les %, em, rem, vw et vh nous permet déjà de créer des mises en page fluides et adaptatives avant même d'utiliser d'autres règles et déclarations.

Contrairement aux pixels (px), elles s'ajustent automatiquement à la taille de l'écran.

```
Ex: .container {  
  width: 80%;  
  font-size: 1.2rem;  
}  
  img {  
    max-width: 100%;  
    height: auto;  
  }
```



2. Media queries

Les media queries permettent d'appliquer des styles CSS spécifiques selon **la taille de nos écrans**.

Elles sont la base du responsive design moderne et c'est pourquoi on se doit de bien les maîtriser.

*Ex : simplifier le header sur mobile (= menu burger)
puis l'étendre sur desktop (= écran d'ordinateur).*

```
@media screen and (max-width: 768px){  
    .menu {  
        display: none; }  
    .burger {  
        display: block;  
    }  
}
```

CSS

```
/* Styles par défaut : mobile */  
.container {  
    width: 100%;  
    font-size: 1rem;  
}
```

```
/* Tablette et plus grand */  
@media screen and (min-width: 768px) {  
    .container {  
        width: 80%;  
        font-size: 1.2rem;  
    }  
}
```

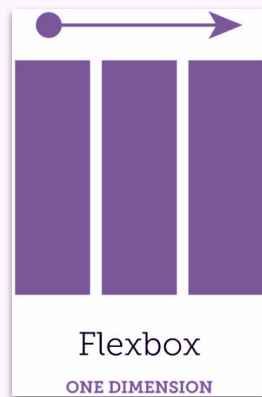
```
/* Desktop Large */  
@media screen and (min-width: 1200px) {  
    .container {  
        width: 70%;  
        font-size: 1.3rem;  
    }  
}
```

3. Flexbox & CSS Grid

Des amis pour la vie

Les systèmes de layout modernes comme **Flexbox** et **CSS Grid** facilitent la création de structures adaptatives sans calculs complexes.

```
/* exemple codepen 📌 */  
  
.container{  
  display: flex;  
  flex-wrap: wrap;  
  }  
  .card {  
    flex: 1 1 300px;  
    margin: 1rem;  
  }
```



VS



4. Les frameworks

Bootstrap & bien d'autres

- Grilles prédéfinies et classes responsive
Ex. : `<div class=" .col-md-6 .col-lg-4">`
Notre élément prendra la moitié de l'écran en format tablette et un rapport de $\frac{1}{3}$ pour les écrans d'ordinateurs
- Composants adaptatifs
Ex. : navbar, cards, boutons, etc.
- Breakpoints standardisés.

5. Navigation & interactions sur mobile

- Menus hamburgers, boutons d'action plus larges.
- Espacements adaptés au tactile.
- Hiérarchisation du contenu et priorisation de l'information.

Exemples : menu déroulant, accordéons, boutons flottants,...

6. Tests & validation responsive

Outils de test :

- Mode responsive de Chrome / Firefox / Wordpress
- Services en ligne pour test & adaptation
([WebsiteResponsive](#), [Responsivetesttool.com](#),
[ResponsiveCheckDesigner](#), [BrowserStack](#), etc.).
- Vérification de la compatibilité entre navigateurs.
([BrowserStack](#), [Can.I.Use](#), [W3C](#))
- Vérification sur support réels (notamment petits smartphones)

Objectif : Un aperçu rapide pour optimiser l'accessibilité et la lisibilité sur différentes tailles d'écran.

7. Bonnes pratiques & tendances actuelles

- Importance du design adaptatif dès la maquette (Figma, XD, etc.).
- Optimisation des performances en rapport avec l'affichage (chargement différé / lazy loading).
- Mise en avant du contenu essentiel sur petit écran
- Design “*fluid typography*” :
[clamp\(\)](#) pour des tailles de police adaptatives
- Dark mode et thèmes variables (UX/UI)

Aller plus loin

- [!\[\]\(a22ba4e13c745edbf29e51af246c4c12_img.jpg\) MDN web docs \(Media Queries\)](#)
(fr, en)

Apprendre en ligne

- [!\[\]\(633dd45d48d71eb51a85c6dd83ee51e9_img.jpg\) OpenClassrooms](#) (fr)
- [!\[\]\(bdddf9191a284aa0945448444083c5b0_img.jpg\) Web.dev](#) (fr, en)
- [!\[\]\(944943bcf87a12c5b9337bf7ed1ef546_img.jpg\) Challenge Front-end](#) (en)
- [!\[\]\(77e1e368d53d3ed6ec2a15bf2432e026_img.jpg\) Frontend Practice](#)

```
<div class="container">
  <div class="carousel-caption">
    <h1>One more for good measure.</h1>
    <p>Cras justo odio, dapibus ac facilisis in, egestas eget quam. Donec id elit
    .</p>
    <p><a class="btn btn-lg btn-primary" href="#" role="button">Browse gallery</a>
  </div>
</div>
div>
v>
class="left carousel-control" href="#myCarousel" role="button" data-slide="prev"
pan class="glyphicon glyphicon-chevron-left" aria-hidden="true"></span>
pan class="sr-only">Previous</span>

class="right carousel-control" href="#myCarousel" role="button" data-slide="next"
pan class="glyphicon glyphicon-chevron-right" aria-hidden="true"></span>
pan class="sr-only">Next</span>

<!-- /.carousel -->

ed Content Section-->

ss="container">
class="row">
div class="col-md-4"></div>
div class="col-md-4"> <h2> FEATURED CONTENT </h2> </div>
div class="col-md-4"></div>
div class="col-md-4"></div>
... 4 small(...)
```